Group 2 - Compiler Project Documentation

How to run

1. Run `make all` to compile the executables named `stage1exe`.
2. Run `./stage1exe <testcase.txt> <output.txt>`. This will open the driver interface as prescribed in the problem statement.

Design Choices and Assumptions

We have listed out all the assumptions and design choices we have made in this section. The functions differ slightly from the given blueprint. This was done to align with our implementation choices.

- The **Lexer** and **Parser** work in parallel, the Parser parses the stream of tokens while the Lexer is analyzing the source code. This is achieved by the Parser calling the Lexer to retrieve the next token tuple from the buffer, rather than storing token tuples in the memory before parsing.
- The **Lexer** tokenizes the comments, but the comment tokens are not parsed by the Parser, as they are not a part of the Grammar.
- The **lexical errors** are also not parsed into the Parser as they are not a part of the Grammar.
- As suggested in the announcements, we have added some keywords (endif, endwhile, etc.) to the **synchronization set** to better **error handling**.

Lexer

The implementation of the **Lexer** is in the following three files:

- `lexerDef.h` : Definitions of the Structures and Enums.
- `lexer.h` : Function declarations.
- `lexer.c` : Function definitions.

lexerDef.h

In this file we have defined the Structures and Enums used for the Lexer. It has the following definitions :

- **`typedef enum token_enum Token`**

These **represent the token IDs** that the Lexer will tokenize the input source code into.

- **`struct tokeninfo`**

This is the **output data type of a token** that has been accepted by the DFA implemented in the `lexer.c` file. It contains the `token`, the corresponding `lexeme`, the `line number`, and a `numValue` field that captures the numeric values if the token is a numeric type (TK_NUM or TK_RNUM).

```
typedef struct tokeninfo{
    Token token;
    char* lexeme;
    int lineNo;
    double numValue;
} TokenInfo;
```

- **`struct twinbuffer`**

This is the buffer used by the Lexer to **store segments of the input source code**. It defines the **buffer as a character array**, maintains the start of the next half of the buffer (`tbStart`), start of the current buffer (`basePtr`), a forward pointer that points to the start of the next lexeme (`forwardPtr`), a file pointer to the input source code (`fp`), and a Boolean flag to indicate if the end of the file has been reached to handle the buffer loading accordingly (`isLastChunk`).

```
typedef struct twinbuffer{
    char buffer[TBSIZE];
    int tbStart;
    int basePtr;
    int forwardPtr;
    FILE* fp;
    bool isLastChunk;
} TwinBuffer;
```

lexer.c

This file implements the functions and logic of the Lexer according to the DFA.

- **`void init(const char* fileName, TwinBuffer* tb)`**

This function initializes the `TwinBuffer` by opening the source file and loading the first chunk into the buffer. It also resets the global line counter.

- **`void getFileStream(TwinBuffer* tb)`**

Used to fill the `TwinBuffer`. It reads a chunk of data from the file and places it in the appropriate half of the buffer. It also handles the end of file condition by appending an `EOF` marker.

- **`char getNextChar(TwinBuffer* tb)`**

Gets the next character from the buffer while maintaining the `forwardPtr`. It automatically calls `getFileStream` when the pointer crosses a buffer boundary.

- **`TokenInfo* getNextToken(TwinBuffer* tb)`**

The overall driver function of the Lexer, it tokenizes the input source code according the the DFA and returns the token tuple. Handles the lexical errors and prints the relevant error message.

- **`Token tokenize<Category>(char* lexeme, TwinBuffer* tb)`**

These functions (`tokenizeRelationalOp`, `tokenizeIDAndKeyword`,`tokenizeNumber`, `tokenizeLogicalOp`, `tokenizeDelimeter` and `tokenizeComment`) implement the state transitions for the various branches of the DFA, called from the main `getNextToken` function. Created for ease of implementation.

- **`TokenInfo* createToken(Token token, char* lexeme, int lineNo, double numValue)`**

Utility function called by `getNextToken` to create a `TokenInfo` structure.

- **`void removeComments(const char* testcaseFile)`**

Utility function to remove comments and print the clean file on terminal.

Parser

The implementation of the Parser is in the following three files:

- `parserDef.h` : Definitions of the Structures and Enums.
- `parser.h` : Function declarations.
- `parser.c` : Function definitions.

parserDef.h

This file contains the data Structures and Enums used by the Parser.

- **`typedef enum{} grammarSymbol`**

Enum **representing both terminals and non-terminals** in our grammar, along with special symbols like `EPSILON`, `DOLLAR`, `SYN` and `PARSER_ERROR`.

- **`struct productionRule`**

This structure stores an individual grammar rule. It includes the Left-Hand Side (LHS) non-terminal and multiple Right-Hand Side (RHS) expansions. `tlen` stores the length of each expansion (For example, if a rule is **A** → **bCD**, then the value of `tlen` for that expansion is 3.), and `hasEPSILON` indicates if the rule can derive epsilon.

```
typedef struct productionRule{
    #define MAXRLEN 10
    #define MAXTLEN 10
    grammarSymbol RHS[MAXRLEN][MAXTLEN];
    int tlen[MAXRLEN];
    grammarSymbol LHS;
    int rlen;
    bool hasEPSILON;
} productionRule;
```

- **`struct Grammar`**

This structure contains the **Grammar rules** and **First and Follow sets**. It includes `rules` that stores all the grammar rules, the `parseTable` for LL(1) parsing, `first` and `follow` Boolean arrays that would contain the **FIRST and FOLLOW** sets of all the Grammar symbols, and Boolean flags like `isFirstComputed` used during set calculation. `parseTableRuleLen` stores the length of the individual Right-Hand Side (RHS) rules present in the `parseTable` and `hasEpsilonFirst` is set to **True** if the **FIRST** set of a Grammar symbol contains **EPSILON**.

```
typedef struct Grammar{
    productionRule rules[NUMRULES];
    grammarSymbol parseTable[NUMSYMBOLS][NUMTOKENS][MAXTLEN];
    int parseTableRuleLen[NUMSYMBOLS][NUMTOKENS];
    bool first[NUMSYMBOLS][NUMTOKENS];
    bool follow[NUMSYMBOLS][NUMTOKENS];
    bool hasEPSILONFirst[NUMSYMBOLS];
    bool isFirstComputed[NUMSYMBOLS];
} Grammar;
```

- **`struct Node`**

Represents **a node in the Parse Tree**. It stores Grammar symbols of the node and its parent along with the token related data (lexeme, line number, numeric value), along with an **array of pointers to its child nodes**.

```
typedef struct Node{
    char* lexeme;
    double numValue;
    grammarSymbol parentSymbol;
    grammarSymbol nodeSymbol;
    int lineNo;
    struct Node* children[MAXTLEN];
    int numChildren;
    bool isLeafNode;
} Node;
```

- **`struct Stack`**

A simple array-based stack used for non-recursive parsing. It stores `Node*` pointers representing subtrees yet to be expanded or matched.

```
typedef struct stack{
    Node* arr[MAXSTACKSIZE];
    int currentSize;
} Stack;
```

parser.c

This file contains the implementation of the Parser's logic, from grammar construction to the parsing driver.

- **`Grammar constructGrammar(char* inputFile)`**

Reads from the input grammar file to populate our `Grammar` structure with production rules.

- **`void calculateFirstSets(Grammar* G)` and `void calculateFollowSets(Grammar* G)`**

Compute the First and Follow sets for each non-terminal, needed to populate the parse table.

- **`void populateParseTable(Grammar* G)`**

Fills the parse table using the first and Follow sets. It also handles synchronization states (`SYN`) for panic mode recovery.

- **`Node constructParseTree(Grammar* G, const char* inputFileName)`**

The main driver that implements the LL(1) parsing algorithm using a stack to build the parse tree.

- **`TokenInfo errorHandler(...)`**

Implements panic mode error recovery, enabling the Parser to continue by synchronizing at `SYN` states.

- **`void printParseTree(Node* curNode, Node* parent, FILE* fp)`**

Prints the constructed parse tree in the specified format (in-order traversal) for analysis

Testcases and Analysis

We have included the sample testcases namely `t1.txt` to `t6.txt`, along with some extra testcases

- `testcase1.txt` : Syntactically correct.
- `testcase2.txt` : Syntactically correct large testcase.
- `testcase3.txt` : Has some lexical errors.
- `testcase4.txt` : Has some syntax errors.
- `testcase5.txt` : Has lexical + syntax errors.

Analysis of selected testcases is provided below, with line by line explanation of specific snippets.

Given testcase `t6.txt`

Input

```
d4cbcd5677<--- ((c3 +             2*c4-(c5-5))/ 4;
c4bbb <--- ((d4cbcd5677- 2.35)*(2345-234*8))+5*c3;
if((~(c4bbb == 0)) && (c4bbb > 78.56))
then
    c6<--- d4cbcd5677/c4bbb
else
    write(c4bbb);
endif
```

Output

```
Line 11 Error : The token TK_SEM for lexeme ; does not match with
the expected token TK_CL
Line no 13 : Error: Unknown pattern <&&>
Line 13 Error: Invalid token TK_OP encountered with value ( stack
top NT_LOGICAL_OP
Line 16 Error: Invalid token TK_ELSE encountered with value else
stack top NT_OPTION_SINGLE_CONSTRUCTED
Line 16 Error: Invalid token TK_ELSE encountered with value else
stack top NT_TERM_PRIME
Line 16 Error: Invalid token TK_ELSE encountered with value else
stack top NT_EXP_PRIME
Line 16 Error : The token TK_ELSE for lexeme else does not match
with the expected token TK_SEM
```

In `Line 11`, there is a missing closing bracket `)` (`TK_CL`) leading to a lexical error, synchronizing at the semicolon `;` (`TK_SEM`) at the end of Line 11.

In `Line 13` we encounter `&&` which is not a valid token, so it is flagged as a lexical error and skipped by the parser. Thus the parser is looking for a relational operator but finds a opening bracket `(` (`TK_OP`) leading to a syntax error as well. The parsing shynchronizes at the end of the line on semicolon.

In `line 15` there is a missing semicolon `;` (`TK_SEM`) at the end of the line, which is flagged as a syntax error by the parser. The code synchronizes at the `else` statement at the start of `Line 16`.

Additional testcase `testcase5.txt`

Input

```
20. % PARSER ERROR INSPITE OF PROPER TOKENIZATION:
21. type int : d4xxxxxxxxxxxxxxxxxxx5 ;
22.
23. read ( b2 ) ;
24.
25. % LEXICAL ERROR: Invalid assignment operator
26. b2 <-- 10 ;
```

Output

```
Line 21 Error: Invalid token TK_FIELDID encountered with value
xxxxxxxxxxxxxxxxxxx stack top NT_GLOBAL_OR_NOT
Line 21 Error: Invalid token TK_NUM encountered with value 5 stack
top NT_GLOBAL_OR_NOT
Line no 26 : Error: Unknown pattern <<-->
Line 26 Error: Invalid token TK_NUM encountered with value 10
stack top NT_OPTION_SINGLE_CONSTRUCTED
Line 26 Error : The token TK_SEM for lexeme ; does not match with
expected token TK_ASSIGNOP
Line 26 Error: Invalid token TK_SEM encountered with value ; stack
top NT_ARITHMETIC_EXPRESSION
```

In `Line 21`, there is a syntax error as `TK_ID` only consists of {`b-d`} or {`2-7`}, which causes the lexer to tokenize b4 and `xxxx...` separately, as `TK_ID` and `TK_FIELDID`. So, the parser gets `TK_FIELDID` instead of `TK_SEM` causing the parser to choose a different rule, which is flagged as a syntax error.

In `Line 26`, lexer flags an unknown pattern error due to a missing hyphen in the assignment operator `<--`. Due to this, the parser did not recieve `TK_ASSIGNOP`, leading to cascading parser errors.

Acknowlegement

Assistance of LLMs was taken for testcase generation and code debugging. All written code is the original work of Group 02 members.

References

- Enums
- Scoped enums
- White spaces removal
- strtod()
- Boolean arrays for set implementation